



中山大學
SUN YAT-SEN UNIVERSITY

数据安全和隐私保护

数据安全和隐私保护第一次实验

姓名 李明昊

学号 24347051

学院 网络空间安全学院

专业 网络空间安全

2026 年 9 月 24 日

目录

1	实验要求	1
2	实验原理	1
2.1	LaTeX 的图片排版	1
2.2	Git 的版本管理模型	1
2.3	Codex 与 CC Switch	2
3	实验设置	2
3.1	软硬件环境	2
3.2	实验仓库与复现方法	2
4	实验过程	4
4.1	LaTeX 图片排版	4
4.2	Git 与 GitHub	4
4.2.1	安装、版本核对与身份配置	4
4.2.2	建立仓库与目录结构	6
4.2.3	两次提交	6
4.2.4	上传到 GitHub 与同步远程更新	7
4.2.5	忽略规则与 Git 配置文件	9
4.3	Codex 的安装、配置与一次任务	9
4.3.1	安装与版本核对	9
4.3.2	登录与基础配置	11
4.3.3	任务一：只读方式阅读仓库	11
4.3.4	任务二：用 Codex 修改代码	12
4.3.5	独立复核修改	15
4.4	CC Switch 的安装与供应商切换	15
4.5	密钥保护	16
5	实验总结	16

1 实验要求

1. 了解 LaTeX 中图片插入的基本方法，并在实验报告中实际插入至少一张实验结果图。实验过程中尽量使用 PDF、SVG、EPS 等矢量图片，避免直接使用低分辨率截图作为论文或实验报告中的结果图；并要掌握 `includegraphics` 的基本使用方法，并能够合理设置图片大小；图片应包含规范的图编号、图标题，并在正文中进行引用。本部分主要作为后续实验报告撰写的基础要求，不要求进行复杂的 LaTeX 排版实践。
2. 掌握 Git 和 GitHub 的基本使用方法，能够完成代码仓库的创建、克隆、提交、上传和更新。学生需安装并配置 Git，然后使用 GitHub 建立个人实验代码仓库，`git push` 本地代码到远程仓库。
3. 完成 Codex 的安装和基本配置，下载并配置 `cc-switch`，对供应商进行配置与切换，并使用 Codex 完成至少一次任务（代码阅读、生成或修改）。
4. 本次实验中，`git`、`codex`、`cc-switch` 之前就已经安装和配置好的同学可以跳过安装和配置过程，仅完成其他部分即可，注意实验报告中别泄露自己的 `api key`。
5. 实验报告需包含必要的操作步骤、关键命令、过程截图、代码修改情况及实验总结。尽量使用 `latex` 模版编写实验报告。

2 实验原理

2.1 LaTeX 的图片排版

位图（PNG、JPG 等）由像素阵列组成，放大后会出现锯齿或模糊；矢量图（PDF、SVG、EPS）保存的是曲线、文字和几何对象，缩放时通常仍能保持清晰。科研图表因此优先导出为 PDF，可减少重复截图造成的清晰度损失；折线图、柱状图、网络结构图等尤其适合直接导出矢量图。

LaTeX 通过 `graphicx` 宏包把图片按版面尺寸排入文档：`\includegraphics[width=...]` 决定图片本身的大小，`\caption` 负责图题并生成图编号，`\label` 记录这个编号，`\ref` 在正文中引用编号。四者配合，使正文论述与图片形成可追溯的引用关系，例如本报告中的图 1 与图 2。

2.2 Git 的版本管理模型

Git 是分布式版本控制系统。本地有三个区域：工作区保存正在编辑的文件，暂存区保存准备提交的变化，本地仓库保存 `commit` 提交历史；此外还有一个位于 GitHub 的远程仓库。它们之间的数据流与常用命令如图 1 所示。

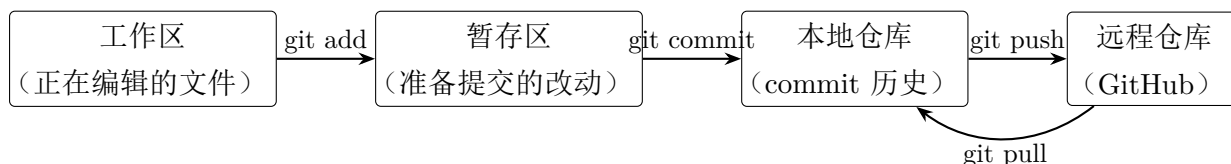


图 1: Git 的工作区、暂存区、本地仓库与远程仓库，以及对应的常用命令

其中 `status` 用于判断当前状态，`diff` 用于比较内容，`log` 用于追溯提交历史，三者都是提交前后的核对工具：哪些文件发生了变化、具体改了什么、准备把哪些变化写入本次提交，都可以先用这三条命令回答。

2.3 Codex 与 CC Switch

Codex 是命令行中的 AI 编程代理。它在用户授权的范围内读取项目、修改文件并运行本地工具，根据自然语言需求和仓库上下文制定并执行操作。由于生成结果仍可能存在错误、遗漏或不安全修改，可靠的使用流程应包含权限确认、命令审阅、程序运行、测试和 Git 差异检查：Codex 负责辅助理解与修改，Git 负责保存可核对的版本。

CC Switch 是第三方开源桌面工具，用于管理 Claude Code、Codex 等命令行工具的供应商配置。它把不同供应商的端点（Endpoint）、模型和认证信息保存为可切换条目：普通切换通过更新 Codex 的配置文件生效，本地代理接管模式则把 Codex 的请求先发送到本机代理再转发给当前供应商，便于热切换、日志和协议转换。无论哪种方式，都必须关注密钥保护、端口暴露和配置恢复问题。

3 实验设置

3.1 软硬件环境

本次实验使用的软硬件版本如表 1 所示。其中 Codex 与 CC Switch 属于实验要求中「已经安装配置好可跳过」的工具，Git 之前也已经安装并完成身份配置，因此本次只补充版本核对与使用记录。

本机没有安装 LaTeX 发行版，报告源码使用课程提供的 SYSUReport 模板编写，在 TexPage 在线平台编译（模板作者的推荐方式），编译引擎为 XeLaTeX。

3.2 实验仓库与复现方法

实验代码仓库为 `research-tools-week1`，目录结构采用实验指导书建议的四类内容：说明文档、代码、结果和报告，如代码清单 1 所示。

```

1 research-tools-week1/
2 |-- README.md           repo overview and how to run the
                           program

```

表 1: 实验环境与工具版本

项目	版本 / 说明
操作系统	Windows (Windows NT 10.0, x86_64)
Git	2.55.0.windows.5
Python	3.14.7 (词频统计程序, 仅标准库)
matplotlib	3.11.2 (仅 <code>--chart</code> 导出矢量图时使用)
Codex CLI	0.156.1 (model: deepseek-flash, provider: custom)
CC Switch	3.20.2 (本机已安装, 本次只做配置与切换核对)
LaTeX	SYSUReport 课程模板 (ctexart 文档类 + XeLaTeX)

```
3 |-- .gitignore                python cache / editor dirs / latex
   temp files / secrets
4 |-- code/
5 |   |-- text_stats.py        the word-frequency program
6 |   `-- sample.txt          sample input text (English passage)
7 |-- result/
8 |   |-- README.md           notes on the run results
9 |   |-- word_freq_report.txt text report produced by the program
10 |   `-- word_freq_top12.pdf result figure (vector PDF)
11 `-- report/                LaTeX lab report
    |-- main.tex
    |-- SYSUReport.cls
14   `-- figures/
```

Listing 1: 实验仓库的目录结构

复现本次实验的全部命令行结果只需要两条命令：第一条生成文本报告与矢量结果图，第二条编译报告。

```
1 # 1) run the program: write both the text report and the vector
   figure
2 python code/text_stats.py code/sample.txt --top 12 \
3     --out result/word_freq_report.txt --chart result/word_freq_top12.
   pdf
4
5 # 2) compile the report with XeLaTeX
6 xelatex main.tex
```

Listing 2: 复现本实验的关键命令

4 实验过程

4.1 LaTeX 图片排版

按实验要求，本部分只完成规范的图片插入，不做复杂排版。具体做法如下。

1. 所有插图统一放在 `report/figures/` 目录下，文件名使用简短的英文与下划线，例如 `word_freq_top12.pdf`。模板的文档类已经用 `\graphicspath` 注册了 `figures/` 等常见插图目录，因此正文中只写文件名即可。
2. 导言区加载 `graphicx` 宏包，正文中用 `figure` 环境插入图片、图题和标签，并用 `width` 参数把图片宽度限制在版心之内。
3. 结果图不使用屏幕截图，而是让统计程序直接把 `matplotlib` 图形保存为 PDF 矢量文件，从根本上避免分辨率损失。
4. 在正文中用「如图 2 所示」的形式引用图片，编译后检查文字、坐标轴和图例是否清晰。

插入图片与正文引用的代码如下，这也是本报告中的实际写法。

```
1 \begin{figure}[htbp]
2   \centering
3   \includegraphics[width=0.8\linewidth]{word_freq_top12.pdf}
4   \caption{Top 12 words in sample.txt}
5   \label{fig:wordfreq}
6 \end{figure}
7 % cite it in the body text: ... as shown in Figure \ref{fig:wordfreq}.
```

Listing 3: 图片插入、图题、标签与正文引用的写法

图 2 是本次实验的实验结果图。语料 `code/sample.txt` 是一段关于数据安全与隐私保护的英文短文，共 257 个词 (tokens)，其中不重复的词 122 个 (types)。图中显示出现次数最多的三个词是 `a(18)`、`data(16)`、`and(12)`：`data` 靠前说明语料主题集中在数据上，而 `a`、`and` 排名靠前是英语虚词本身高频导致的，这也说明纯词频统计需要配合停用词表才有更强的语义解释力。该图由程序直接导出为 PDF 矢量文件，图内文字与条形边缘在任意缩放下都保持清晰，与使用屏幕截图相比没有锯齿。

4.2 Git 与 GitHub

4.2.1 安装、版本核对与身份配置

本机的 Git 之前已经安装，本次先核对版本，再确认提交记录中使用的姓名与邮箱。

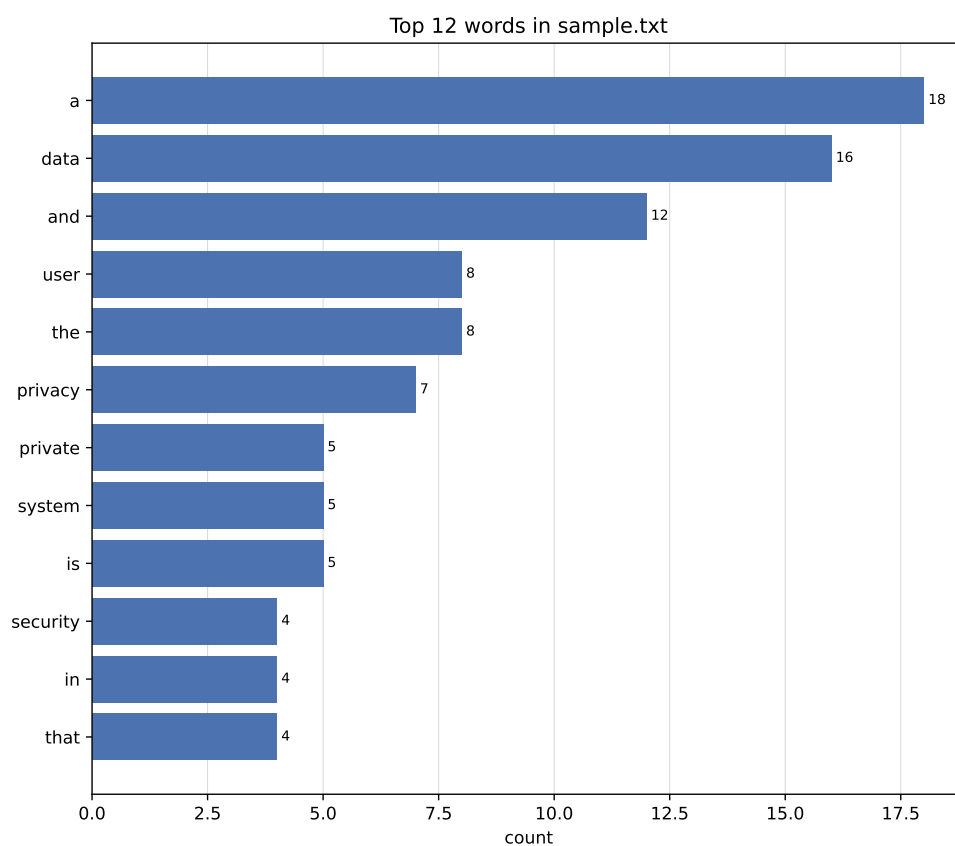


图 2: sample.txt 中前 12 个高频词的出现次数（由程序直接导出的矢量 PDF）

```
1 git --version
2 # git version 2.55.0.windows.5
3
4 git config --global user.name "limh253"
5 git config --global user.email "leeminghao791@gmail.com"
6 git config --global --list --show-origin
7 # file:C:/Users/13405/.gitconfig core.editor="D:\Microsoft VS Code\
   bin\code" --wait
8 # file:C:/Users/13405/.gitconfig user.name=limh253
9 # file:C:/Users/13405/.gitconfig user.email=leeminghao791@gmail.com
```

Listing 4: 核对 Git 版本与全局身份配置

--show-origin 会显示每一项配置来自哪个文件,本机结果为 C:/Users/13405/.gitconfig。姓名和邮箱只是写入提交历史,并不等同于 GitHub 登录密码;这条命令也用于检查配置文件中是否误存了密码或访问令牌,本次检查结果是配置中只有编辑器与身份信息,没有密钥。

4.2.2 建立仓库与目录结构

实验指导书给出的流程是在 GitHub 上新建仓库后 `git clone` 到本地。本实验采用先本地初始化、再关联远程仓库的方式,原因是实验代码已在本地写好,本地建仓可以避免远程 README 与本地首次提交产生无关历史 (unrelated histories) 冲突。

```
1 git init -b main
2 git status --short
3 # ?? .gitignore
4 # ?? README.md
5 # ?? code/
```

Listing 5: 本地初始化仓库并确认初始状态

仓库中建立 README.md、code/、result/、report/ 四类内容。需要注意的是空目录不会被 Git 记录,因此 result/ 中放入说明文件 README.md 后才随提交进入版本库,目录结构见代码清单 1。

4.2.3 两次提交

每次提交前依次回答三个问题: 哪些文件发生了变化 (status), 具体改了些什么 (diff), 准备把哪些变化写入本次提交 (add 之后看 `diff --staged`)。初学阶段明确列出要暂存的文件,而不是在不了解差异时直接使用 `git add .`; 一次提交只完成一个清晰目标。

第一次提交加入 README 和可运行的小程序；第二次提交补充 Codex 新增的功能与运行结果。在此之后，本实验报告本身作为第三次提交纳入版本管理（后续刷新过程记录图的提交同理），这样每次提交只完成一个清晰目标。前两次提交的实际终端记录如图 3 所示，关键命令如下。

```
1 git status --short
2 git add README.md .gitignore code/text_stats.py code/sample.txt
3 git diff --staged --stat
4 git commit -m "add README and a runnable word-frequency program"
5
6 # second commit: the --out/--chart options plus the run results
7 git add code/text_stats.py README.md result/
8 git diff --staged --stat
9 git commit -m "add out/chart options to text_stats and record the
   week-1 run results"
10
11 git log --oneline --decorate
```

Listing 6: 两次提交使用的关键命令

4.2.4 上传到 GitHub 与同步远程更新

在 GitHub 上新建仓库 `research-tools-week1`（选择空仓库，不初始化 README），然后关联远程地址并上传。第一次上传使用 `-u` 建立上游关系，之后直接使用 `git push` 即可。

```
1 git remote add origin https://github.com/<username>/research-tools-
   week1.git
2 git branch -M main
3
4 # this machine cannot reach github.com directly, so the push goes
   through the
5 # local proxy on 127.0.0.1:7890; -c applies to this one command only
6 git -c http.proxy=http://127.0.0.1:7890 -c https.proxy=http
   ://127.0.0.1:7890 \
7     push -u origin main
8
9 git remote -v
10 git log --oneline --graph --decorate -n 10
```

Listing 7: 关联远程仓库并上传本地提交

```

PS> git status --short
?? .gitignore
?? README.md
?? code/

PS> git add README.md .gitignore code/text_stats.py code/sample.txt

PS> git status --short
A .gitignore
A README.md
A code/sample.txt
A code/text_stats.py

PS> git diff --staged --stat
.gitignore      | 24 ++++++
README.md       | 39 ++++++
code/sample.txt | 15 ++++++
code/text_stats.py | 85 ++++++
4 files changed, 163 insertions(+)

PS> git commit -m "add README and a runnable word-frequency program"
[main (root-commit) 2ab2d82] add README and a runnable word-frequency program
4 files changed, 163 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 code/sample.txt
create mode 100644 code/text_stats.py

PS> git status --short
M README.md
M code/text_stats.py
?? result/

PS> git add code/text_stats.py README.md result/word_freq_report.txt result/word_freq_top12.
pdf result/README.md

PS> git diff --staged --stat
README.md      | 9 +++++
code/text_stats.py | 65 ++++++
result/README.md | 28 ++++++
result/word_freq_report.txt | 18 ++++++
result/word_freq_top12.pdf | Bin 0 -> 12815 bytes
5 files changed, 119 insertions(+), 1 deletion(-)

PS> git commit -m "add out/chart options to text_stats and record the week-1 run results"
[main 5a66dd4] add out/chart options to text_stats and record the week-1 run results
5 files changed, 119 insertions(+), 1 deletion(-)
create mode 100644 result/README.md
create mode 100644 result/word_freq_report.txt
create mode 100644 result/word_freq_top12.pdf

PS> git log --oneline --decorate
5a66dd4 (HEAD -> main) add out/chart options to text_stats and record the week-1 run results
2ab2d82 add README and a runnable word-frequency program

```

图 3: 两次提交的终端记录: status 查看变化、add 明确暂存、diff -staged 复核、commit 提交

本机直连 github.com 会出现连接超时 (Failed to connect to github.com port 443), 而本机 7890 端口已有代理进程在运行, 因此推送时用 `-c http.proxy` 临时指定代理; 这也印证了实验指导书中关于 `http.proxy` 的说明: 只在确实需要时才设置代理, 用完及时取消, 并且不要把代理地址与令牌一起写死进仓库。首次通过 HTTPS 上传时, Git Credential Manager 会在浏览器或凭据管理器中要求完成 GitHub 认证, 认证信息由系统凭据管理器保存, 不应写入脚本或仓库。

实验指导书还要求验证一次「远程更新」流程: 在 GitHub 网页上直接修改 README.md 的一行文字并提交 (这次提交只存在于远程), 再在本地执行 `git pull` 把远程提交合并到当前分支, 最后用 `git log` 核对本地提交与远程更新, 命令如下。需要说明的是, 本次实验只完成了上传, 没有执行这一步远程更新, 因此图 4 中看到的提交历史全部来自本地, `origin/main` 与本地 `main` 指向同一个提交; 网络通畅时按下面的命令补做即可。

```
1 git status          # make sure the working tree is clean first
2 git pull
3 git log --oneline --graph --decorate -n 10
```

Listing 8: 同步远程更新并核对历史

远程仓库信息、提交历史与工作区状态记录如图 4 所示。实践中可以形成固定节奏: 开始工作先 `pull`, 修改后看 `status` 和 `diff`, 确认后 `add`、`commit`, 结束工作再 `push`。

4.2.5 忽略规则与 Git 配置文件

`.gitignore` 中排除了 Python 缓存 (`__pycache__`)/、虚拟环境、编辑器目录、LaTeX 中间产物 (保留 `main.pdf` 便于查阅), 以及 `.env` 和 `*.key` 等可能保存密钥的文件。Git 的用户级配置文件是用户主目录下的 `.gitconfig`, 应优先使用 `git config` 命令修改而不是手工编辑, 避免格式错误; 仅当所在网络确实需要代理时才设置 `http.proxy` (示例端口 7890 应替换为本机代理软件显示的实际地址), 不需要时用 `git config --global --unset http.proxy` 及时取消。

4.3 Codex 的安装、配置与一次任务

4.3.1 安装与版本核对

Codex CLI 之前已经安装, 本次先核对版本:

```
1 codex.cmd --version
2 # codex-cli 0.156.1
```

Listing 9: 核对 Codex CLI 版本

这里有一个本机的实际问题值得记录: Codex 通过 npm 安装后会生成 `codex.ps1`、`codex.cmd`、`codex` 三个入口, 而本机 PowerShell 的执行策略为 `Restricted`, 直接运行

```

PS> git remote -v
origin    https://github.com/limh253/research-tools-week1.git (fetch)
origin    https://github.com/limh253/research-tools-week1.git (push)

PS> git log --oneline --graph --decorate --all
* 5a7b7bf (HEAD -> main, origin/main) record the final pushed history in the report figure
* 866d1a1 align the report wording with the final git history
* de03cde refresh the git log record figure after pushing to GitHub
* 7861ef6 add the LaTeX report for experiment 1 (figures, workflow diagram and process records)
* 5a66dd4 add out/chart options to text_stats and record the week-1 run results
* 2ab2d82 add README and a runnable word-frequency program

PS> git status --short

PS> git show --stat 5a66dd4
commit 5a66dd478e73d997221fd4e526bd676474766e29
Author: limh253 <leeminghao791@gmail.com>
Date: Thu Sep 24 19:39:10 2026 +0800

    add out/chart options to text_stats and record the week-1 run results

 README.md | 9 ++++++
 code/text_stats.py | 65 ++++++++++++++++++++++++++++++++++++++
 result/README.md | 28 ++++++
 result/word_freq_report.txt | 18 ++++++
 result/word_freq_top12.pdf | Bin 0 -> 12815 bytes
 5 files changed, 119 insertions(+), 1 deletion(-)

```

图 4: 远程仓库信息、提交历史与工作区状态 (origin/main 与本地 main 一致)

codex 会因为「在此系统上禁止运行脚本」而失败。解决办法有两条：改用 `codex.cmd` 调用，或把执行策略调整为 `RemoteSigned`。本实验使用前者，不修改系统安全设置。

4.3.2 登录与基础配置

Codex 的用户级配置是用户主目录下的 `.codex/config.toml`。本机使用 API Key 方式登录第三方供应商，与供应商、模型和权限相关的关键字段如下（按要求隐去密钥）。

```
1 model_provider = "custom"
2 model = "deepseek-flash"
3 model_reasoning_effort = "high"
4
5 [model_providers.custom]
6 name = "custom"
7 wire_api = "responses"
8 requires_openai_auth = true
9 base_url = "https://api.deepseek.com/v1" # the API key lives in the
    OS credential store, never in the repo
10
11 [windows]
12 sandbox = "elevated"
```

Listing 10: Codex 配置中与供应商、模型和权限相关的字段（密钥已隐去）

`base_url` 指向供应商的端点，密钥通过环境变量或系统凭据保存，不写入仓库，也不出现在本报告的截图与配置摘要中。在 Codex 会话中，`/status` 查看当前目录、模型与权限，`/model` 切换模型与推理强度，`/permissions` 调整命令与文件访问权限，`/quit` 退出。

4.3.3 任务一：只读方式阅读仓库

第一个任务是让 Codex 只读地阅读仓库，说明目录结构、程序入口和运行方法。这里使用 `codex exec` 的非交互模式，并用 `-s read-only` 把沙盒限制为只读，从权限层面保证它不会修改任何文件；给它的自然语言指令是「请先阅读这个项目，说明目录结构、程序入口、运行方法，以及每个文件的作用，不要修改任何文件」。

```
1 codex exec -s read-only "<natural-language_task_prompt>"
2 # <...> = the Chinese instruction above; -s read-only limits the
    sandbox to read-only
```

Listing 11: Codex 只读阅读任务的调用形式

本次会话的终端记录如图 5 所示。Codex 自己发起了多次只读命令（`git log/git status`、列目录、读 README 与源码，并实际运行了一次程序），再给出结论，其中关键结论摘录如下。

已经读完整仓库（未做任何修改，`git status` 保持干净）。程序入口是 `code/text_stats.py:84`，`tokenize()` 按正则 `WORD_RE` 切词（允许 `don't`、`multi-party` 这类词，默认转小写），再由 `count_words()` 用 `Counter` 计数、`build_report()` 生成对齐的排行文本；只依赖标准库，没有安装步骤。它还指出仓库中有三处与文档不一致、需要确认：README 提到的 `report/main.tex` 尚不存在、`result/` 为空、本地仓库还没有配置远程地址。

这几处提示都是有价值的：它说明 AI 的阅读结论可以被独立核对（例如它给出的入口行号与文件实际内容一致），也说明「只读任务」适合用来在动手修改之前先建立对项目的正确认识。

4.3.4 任务二：用 Codex 修改代码

第二个任务明确而具体：给词频统计程序增加 `--out`（把文本报告写入文件）和 `--chart`（导出前 N 个高频词的矢量 PDF 柱状图）两个选项，要求保持原有默认行为和终端输出格式不变，并同步更新 README 的用法示例，改完后自行运行验证。

```
1 codex exec -s workspace-write "<natural-language_task_prompt>"
2 # <...> = the Chinese instruction above; -s workspace-write allows
   edits in the workspace
```

Listing 12: Codex 代码修改任务的调用形式

本次使用 `-s workspace-write`，允许它在工作区内写文件（仍然不能访问工作区之外）。Codex 的修改内容如下。

- 新增函数 `write_report(report, out)`：把同一份文本报告写入 `--out` 指定的文件，父目录用 `mkdir(parents=True, exist_ok=True)` 自动创建，写失败时打印错误并返回失败状态。
- 新增函数 `build_chart(path, counter, top, chart)`：用 `matplotlib` 画前 N 个高频词的横向条形图，条端标注次数，`savefig(..., format='pdf')` 输出矢量 PDF；`matplotlib` 采用函数内延迟导入，不带 `--chart` 时完全不加载。
- `parse_args()` 中新增 `--out` 与 `--chart` 两个参数，`main()` 中在打印报告之后按需写文件、画图，任一步失败则返回非零状态。
- README.md 的「运行方法」一节补充两条示例，并说明两个选项都不改变默认行为。

```

OpenAI Codex v0.156.1
-----
workdir: C:\Users\13405\Desktop\research-tools-week1
model: deepseek-flash
provider: custom
approval: never
sandbox: read-only
reasoning effort: high
reasoning summaries: none
..... (中间的 exec 调用过程省略) .....
exec
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command 'git log --online
-20; git status --short' i
n C:\Users\13405\Desktop\research-tools-week1
exec
"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -Command 'Get-ChildItem -Re
cursive -Force | Select-Objec
t FullName, Length, Mode | Format-Table -AutoSize' in C:\Users\13405\Desktop\research-tools-
week1
succeeded in 749ms:
2ab2d82 add README and a runnable word-frequency program
.....
已读完整仓库（未做任何修改，`git status` 保持干净）。
**目录结构**
...
research-tools-week1/
├── .gitignore          忽略规则（Python 缓存、编辑器目录、LaTeX 中间产物、密钥）
├── README.md          仓库说明与运行方法（UTF-8）
├── code/
│   ├── text_stats.py  词频统计程序，唯一的可执行入口
│   └── sample.txt     程序输入样例，15 行英文短文（数据安全/隐私主题）
├── result/           预期存放运行结果，当前为空
└── report/
    └── figures/       预期存放 LaTeX 插图，当前为空
...
**程序入口**
- 入口是 `code/text_stats.py:84` 的 `if __name__ == '__main__':`，调用 `main()`（`code/text_
stats.py:67`）。
- 数据流：`parse_args()`（`:56`）解析参数 → 读取文件 → `tokenize()`（`:24`）按正则 `WORD_RE`
（`:21`，允许 `don't`、`multi-party` 这类词）切词并默认转小写 →
`count_words()`（`:35`）用 `Counter` 计数 → `build_report()`（`:40`）生成对齐的排行文本并通
过 `print` 输出。
- 只依赖标准库（`argparse`、`re`、`collections`、`pathlib`），无安装步骤。
**运行方法**
```bash
python code/text_stats.py code/sample.txt # 默认 top 10

```

图 5: Codex 只读阅读任务的会话记录（会话头信息、自主执行的只读命令与部分结论）

```

1 parser.add_argument('--out', type=Path, metavar='FILE', default=
 None,
2
3 help='also write the text report to FILE (
 parent directories are created)')
4 parser.add_argument('--chart', type=Path, metavar='FILE', default
 =None,
5
6 help='write a bar chart (PDF) of the top N
 words to FILE')

```

Listing 13: 新增的命令行参数（节选）

```

1 def build_chart(path: Path, counter: Counter, top: int, chart: Path)
 -> bool:
2 # plot the top-N words as a horizontal bar chart and save it as a
 vector PDF
3 try:
4 import matplotlib
5 matplotlib.use('Agg') # no-GUI backend, required on
 this machine
6 import matplotlib.pyplot as plt
7 except ImportError:
8 print('error: --chart needs matplotlib, install it with: pip
 install matplotlib')
9 return False
10
11 items = counter.most_common(top)
12 fig, ax = plt.subplots(figsize=(8, 0.45 * len(items) + 1.6))
13 ax.barh(positions, counts, color='#4c72b0')
14 ...
15 chart.parent.mkdir(parents=True, exist_ok=True)
16 fig.savefig(chart, format='pdf')

```

Listing 14: 新增的矢量图导出函数（节选）

Codex 在总结中提到了一处它自己发现的环境问题：本机 matplotlib 默认选择 Tk-Agg 后端，而该环境的 Tcl 不完整，plt.subplots() 会直接抛出 TclError，因此它把后端显式固定为无界面的 Agg，保证在无 GUI 环境下也能稳定出图。这属于合理的工程判断。

### 4.3.5 独立复核修改

按可靠流程，不能只接受 AI 的自我报告，必须用 Git 与运行结果独立复核。本次复核做了四件事。

```
1 git status --short # only the two expected files should be
 modified
2 git diff --stat
3 git diff -- README.md # read the changes line by line
4
5 # actually run it and check the behaviour
6 python code/text_stats.py code/sample.txt --top 12 \
7 --out result/word_freq_report.txt --chart result/word_freq_top12.
 pdf
8 # exit code 0
9
10 # confirm the exported figure is really vector, not a raster image
11 # file header %PDF- , no /Subtype /Image object, contains /Font
```

Listing 15: 用 Git 与运行结果复核 Codex 的修改

复核结果：`git status` 显示只有 `README.md` 与 `code/text_stats.py` 被修改，没有多改文件；程序返回码为 0，终端输出的格式与修改前完全一致，`result/` 下正常生成了文本报告与 PDF；再检查 PDF 文件头为百分号加 PDF-、内部不含 `/Subtype /Image` 位图对象、包含 `/Font` 字体对象，确认它是真正的矢量图而不是被栅格化的图片。确认无误后再提交并上传：Codex 负责辅助修改，Git 负责保存可核对的版本，这正是实验要求的工作方式。

## 4.4 CC Switch 的安装与供应商切换

CC Switch 用于管理 Codex 等命令行工具的供应商配置。本机之前已经安装 CC Switch（版本 3.20.2），按实验要求第 4 条，已经安装配置好的工具可以跳过安装步骤，因此本次只做配置核对与供应商切换的验证。

本机 CC Switch 的应用数据目录是用户主目录下的 `.cc-switch/`：其中 `cc-switch.db` 保存供应商条目（`providers` 表里同时有 Codex 的默认登录项和自定义供应商，还有 `provider_endpoints`、`model_pricing` 等表），`settings.json` 记录界面语言、是否启用本地代理以及当前选中的 Codex 供应商，`backups/` 与 `logs/` 分别保存数据库备份和运行日志。CC Switch 切换供应商时写入的，正是 Codex 配置文件中的 `[model_providers.*]` 段，也就是代码清单 10 里 `custom` 供应商的来源；如果启用本地代理接管模式，请求会先经过 `proxy_config` 表中记录的本地监听地址与端口再转发给供应商。

需要核对与切换时的操作步骤如下。

1. 打开 CC Switch，切换到 Codex 页面，点击添加供应商；优先选择内置预设。若使用自定义供应商，则按供应商文档填写名称、API Key（或认证信息）、Endpoint URL 与模型名，Codex 的端点地址通常以 /v1 结尾。
2. 保存后点击供应商卡片上的「启用」，或在系统托盘菜单中选择要使用的供应商，切换会更新 Codex 的配置文件。
3. 关闭并重新打开 Codex CLI 或 Codex 应用，使新的供应商与模型列表生效，然后在会话中用 `/status` 查看当前模型与权限，并用一个小问题验证调用是否正常。

使用第三方供应商之前，应了解其计费方式、数据处理规则和账号规则；本地代理接管模式会多出一层本地服务，因此还要关注端口暴露与配置恢复的问题。

## 4.5 密钥保护

实验要求特别提示不要在报告中泄露 API Key。本次实验采取了以下措施：密钥只保存在 Codex 的配置文件或系统凭据中，不写入仓库；`.gitignore` 中排除了 `.env`、`*.key` 等文件；本报告中的所有配置摘要与过程记录图都只保留 `base_url`、模型名等非敏感字段，涉及密钥的位置一律省略；同时用 `git config --global --list --show-origin` 检查了 Git 配置中没有误存令牌。

## 5 实验总结

本次实验围绕科研中的基础工具链完成了三部分练习，收获主要有以下几点。

第一，图片规范方面，我真正区分了位图与矢量图的差别，并实践了「不截图、让程序直接导出矢量图」的做法。`matplotlib` 保存为 PDF 之后，图内文字与条形边缘在放大时依然清晰，比截屏贴图更可靠；同时掌握了 `\includegraphics` 的宽度控制，以及 `caption-label-ref` 这一套图题与引用的配合方式。

第二，版本管理方面，最有价值的不是记住几条命令，而是形成固定的核对节奏：开始工作先 `pull`，改完先看 `status` 和 `diff`，暂存时明确列出文件名，提交信息用动词描述本次变化，结束工作再 `push`。实验中我还踩到了两个具体问题：一是空目录不会被 Git 记录，需要在目录中放一个说明文件；二是「先本地建仓、再关联远程」可以避免远程 README 与本地首次提交之间的无关历史冲突。

第三，AI 辅助编程方面，最深的体会是「AI 负责理解与修改，人负责确认与版本记录」。让 Codex 用只读沙盒先读项目，它指出了 README 与仓库实际内容的三处不一致；再让它用 `workspace-write` 沙盒完成一个明确的小任务，它主动发现了 TkAgg 后端在本机不可用的问题并改用 Agg 后端。但我没有直接相信它的自我报告，而是用 `git status` 和 `git diff` 逐行核对改动，再实际运行程序、检查返回码和输出格式，最后还

用文件头与内部对象类型确认导出的确实是无损矢量图。这个「权限确认-命令审阅-运行验证-差异检查」的流程，比单纯让 AI 写代码更有意义。

此外，这次实验也让我注意到几个容易忽略的细节：PowerShell 的执行策略会阻止 npm 生成的 .ps1 入口，需要用 .cmd 入口或调整策略；Windows 控制台默认编码与 UTF-8 不一致，程序输出使用 ASCII 标签可以避免中文乱码；以及密钥绝不能进入仓库、报告和截图。实验报告本身也用同一个 Git 仓库管理，代码、结果与报告放在一起，便于后续实验复用这套流程。